

Einführung

Programmierprojekt Timo Stadler

Heute

- Allgemeines zum Vorgehen
- Bewertung
- Vorstellung der Projektideen

Programmierpraktikum

- Ziel: Sie lernen, ein Projekt umzusetzen
 - Feinplanung des Projektziels, der Deliverables
 - Planung der Vorgehensweise
 - Durchführung (=hauptsächlich programmieren)
 - Reporting

Arbeitsmittel

- eine Programmieraufgabe in C oder C++
- angemessene Dokumentation entsprechend den *Best Practices*
- **Präsentationen** mit (beliebigem) Werkzeug (Markdown, PPT, LaTeX)
- Versionierung mit **GIT** (darüber sprechen wir noch) zwingend (ermöglicht mir, Ihren Projektfortschritt zu tracken)
- Einsatz eines **Build-Management-Tools** (make)
- Ggfs. Debugging mit (graphischem) **Debugger** (GDB)
- Ggfs. **Containerisierung** der Anwendung (Docker)

Vorgehen I

- Sie arbeiten zu zweit (nur in Ausnahmefällen alleine
 - oder: Sie arbeiten in einer Gruppe an einem etwas größeren Projekt mit klar abgegrenzten Teilprojekten
 - Sie wählen einen Projektvorschlag oder stellen mir Ihren eigenen vor
 - Sie bearbeiten das Projekt
 - Sie stellen die Ergebnisse und das aus Ihrer Sicht Interessante vor
 - ich stehe Ihnen bei den Schritten zur Seite und unterstütze
-
- (sehr viel Sie, sehr wenig ich)

Vorgehen II

- verpflichtende gemeinsame Treffen
 - (heute, in 2 Wochen, Live-Coding Session, Abschluss)
- Heimarbeit (sehr viel!)
 - 2 SWS $\triangleq$ 3 CP $\triangleq$ 90h
- regelmäßige Fragestunde/Sprechstunde
- Intro-Vorlesungen zu den geforderten Tools (Git, Docker, Linux, ...)
- Möglichkeit zur begleiteten Arbeit
- Abschlusspräsentation und Abschlussposter

Bewertung

- Ihr Software-Produkt (60%)
- Live-Coding Session(20%)
 - Notwendig um Verwendung von Gen-AI auszuschließen
 - Jedes Team erhält zu Beginn ein neues Feature, das es umzusetzen gilt
 - 1,5h Zeit
 - Ich werde in dieser Zeit unterstützen aber auch kritische Fragen zum Code stellen
- Abschlusspräsentation (20%)
- meine Wahrnehmungen über den Arbeitsprozess

Projektideen

- es soll etwas Neues, mit Pfiff für Sie dabei sein
 - oft “nerdig”, das macht aber nichts
- es basiert auf dem aus PG1 (und PG2) bekannten Wissen
- Verwendung und Einbindung von Librarys für Ihr Projekt ist explizit erwünscht (Boost, Ncurses, ...)
- **Restrictions:**
 - **Keine** Verwendung einer „Game-Engine“ (Unreal, Godot, ...)
 - **Kein** ML/KI Projekt (Projekt soll deterministischen Ansatz verfolgen)
 - CI/CD ist nicht für das Projekt notwendig

Projektideen (Routeplanning)

- **Interaktiver Search-Space Visualizer:**
 - Eine Visualisierung, die die Funktionsweise von A*, Dijkstra und BFS auf Rasterkarten Schritt für Schritt animiert.
- **Graph-Based Railway Scheduler:**
 - Ein CLI-Tool zur Verwaltung eines Schienennetzes, das Zugkonflikte auf eingleisigen Strecken mittels Graphenalgorithmien erkennt und Ausweichrouten berechnet.
- **Offline OpenStreetMap Pathfinder:**
 - Ein Program, das .osm-XML-Dateien einliest, ein kompaktes Adjazenzlisten-Netzwerk im RAM aufbaut und Routenberechnungen durchführt
- **Virtual Traffic Flow Simulator:**
 - Eine 2D-Simulation von Strassenkreuzungen, bei der Autos als Partikel einem Graphen folgen und Staudynamiken (Cellular Automata) visualisieren.
- **GTFS Public Transport Route Finder:**
 - Ein C++ Tool, das reale ÖPNV-Fahrpläne im GTFS-Format parst.
- **Dynamic Vehicle Routing (VRP) Solver:**
 - Ein C++ Konsolenprogramm, das Näherungslösungen für VRP mit Kapazitätsgrenzen berechnet.

Projektideen (Computer Vision)

- **Shape & Contour Classification Engine:**
 - Eine Anwendung, die geometrische Formen und Objekte in Bildern klassifiziert und deren Schwerpunkt sowie Ausrichtung bestimmt.
- **Real-time Color & Motion Tracker:**
 - Eine Anwendung, die bewegte Objekte anhand von HSV-Farbräumen im Webcam-Stream nachverfolgt und Trajektorien zeichnet.
- **ASCII-Art Video Stream Converter:**
 - Eine Anwendung, die ein Bild analysiert und in ein strukturiertes ASCII-Art-Format wandelt.
- **Optical QR / Barcode Scanner Pipeline:**
 - Eine Bildverarbeitungs-Pipeline zur Dekodierung von 1D-Barcodes aus Bildern.
- **Feature-Based Panorama Stitcher:**
 - Eine Anwendung, die zwei überlappende Fotos analysiert, markante Punkte (z. B. SIFT/ORB) findet und zu einem Panorama zusammenfügt.
- **Face-Tracking und –anonymisierung**
 - Sie nutzen OpenFace zur Gesichtserkennung, anonymisieren das Gesicht (Unschärfe) und fügen stattdessen OpenFace-Merkmale hinzu
- **Face-Feature-Classification**
 - Sie nutzen oder entwickeln ein Verfahren, welches für Gesichter unterschiedliche Eigenschaften (schaut links, lächelt, nickt) erkennt

Projektideen (etc und Audio)

- **Schach und Schachcomputer (oder andere Spiele)**
 - Schach-Implementierung (Regeln prüfen, ...) + 2-Personen-Spiel
- **Turing-Maschine**
 - Sie konstruieren eine Turing-Maschine in C (endl. Automat + unendliches Band) auf dieser Basis bauen Sie einen brainfuck-Interpreter
- **Langzahlarithmetik**
 - Typischerweise unterstützen Computer beschränkte Zahlenräume (32- oder 64 bit). Natürlich können auch beliebig lange Zahlen repräsentiert werden, zum Beispiel mittels Listen. Damit könnte man dann Tupper's Formula berechnen.
- **Conway's Game of Life**
 - kleiner Regelsatz welche "Zellen" eines (im Prinzip) unendlichen Feldes abhängig von ihren Nachbarn belebt werden bzw. absterben
 - Sie implementieren und vergleichen Variationen des Game of Life, u.a. für Hexagon-basierte Welten.
- **hardware-/systemnahe Soundkarten-Programmierung**
 - Sie implementieren Programme zur Ein-/Ausgabe mit minimaler Verzögerung und messen diese Verzögerung
- **Vocoding**
 - Sie nutzen einen Vocoder um Stimmen zu verfremden (in Fast-Echtzeit)
- **Beat Detection & BPM Estimator:**
 - Ein Algorithmus, der Rhythmusmuster in Musikdateien analysiert und das Tempo (BPM) schätzt.
- **Software Synthesizer Engine (Additive & Subtractive)**
 - Eine flexible Synth-Engine, die Oszillatoren (Sinus, Sägezahn), Filter und ADSR-Hüllkurven zur Live-Klangsynthese bereitstellt.
- **Real-time Spectrum Analyzer & Equalizer:**
 - Ein Tool, das Audio-Streams per (FFT) in Frequenzbänder zerlegt, und DSP-Filter (Low-pass) anwendet.